Course Discussions

Solution: Remove All Adjacent Duplicates In String

Problem Statement

Given a string `S`, remove all adjacent duplicate characters recursively to generate the resultant string.

Examples

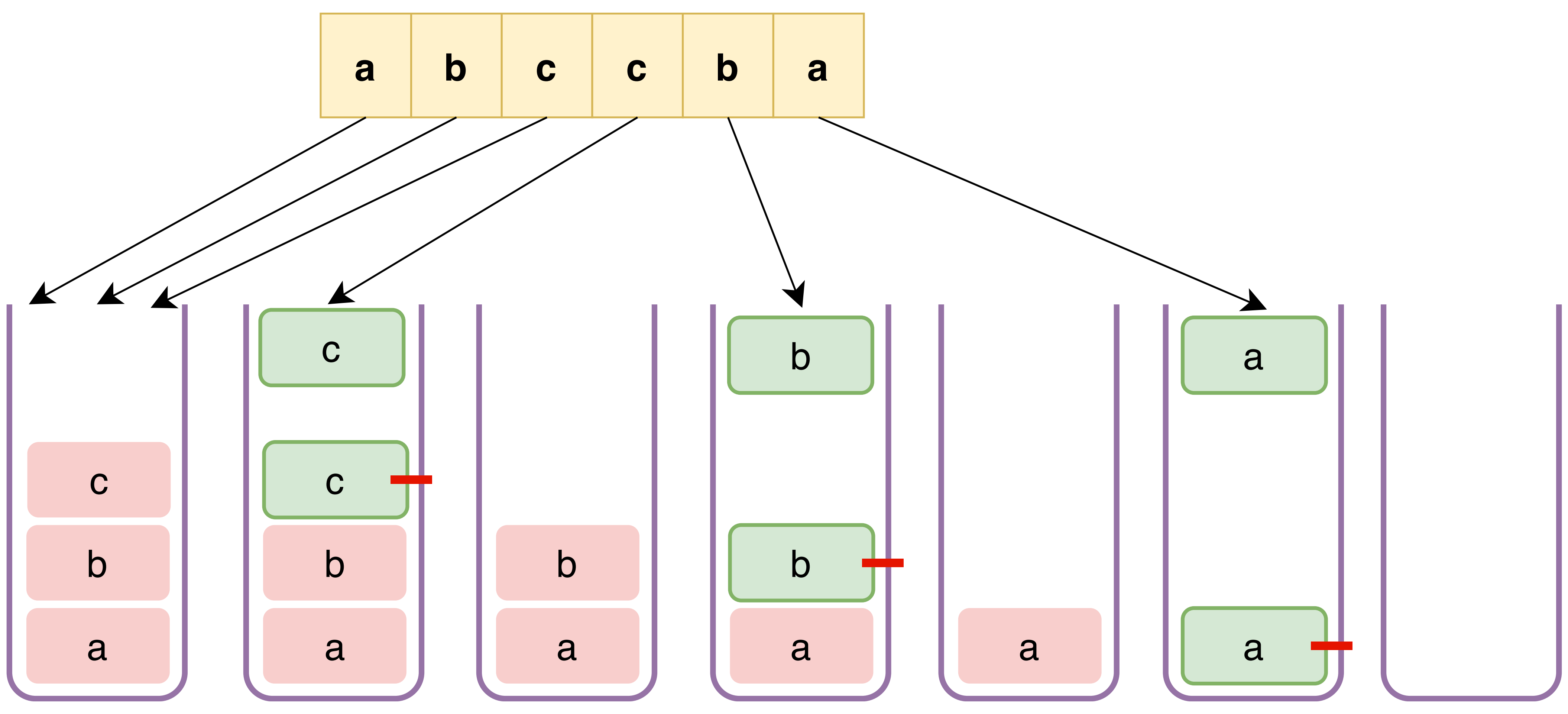
1. - Input: `s = "abccba"`
   - Output: `""`
   - Explanation: First, we remove "cc" to get "abba". Then, we remove "bb" to get "aa". Finally, we remove "aa" to get an empty string.
2. - Input: `s = "foobar"`
   - Output: `"fbar"`
   - Explanation: We remove "oo" to get "fbar".
3. - Input: `s = "abcd"`
   - Output: `"abcd"`
   - Explanation: No adjacent duplicates so no changes.

Solution

This problem can be solved efficiently using a stack, which can mimic the process of eliminating adjacent duplicates.

Algorithm Walkthrough

1. Initialize an empty stack.
2. Loop through the characters in the given string `s`.
3. For each character:
   - If the stack is not empty and the current character is the same as the top character on the stack, pop the character from the stack.
   - Otherwise, push the current character onto the stack.
4. Finally, build the result string from the characters remaining on the stack.



Code

Here is the code for this algorithm:

Java

```java
public class Solution {
    public String removeDuplicates(String s) {
        // Create a StringBuilder to use as a stack
        StringBuilder stack = new StringBuilder();

        // Process each character in s
        for (char c : s.toCharArray()) {
            int length = stack.length();
            // If the stack is not empty and the current character is the same as the to
            if (length > 0 && c == stack.charAt(length - 1))
                stack.deleteCharAt(length - 1);
            else // Push the current character onto the stack
                stack.append(c);
        }

        // Convert the stack to a string
        return stack.toString();
    }

    public static void main(String[] args) {
        Solution solution = new Solution();
```

Run Code Result Stdout

You must run your code first

Execute

Time and Space Complexity

The time complexity of this algorithm is $O(N)$, where N is the length of `s`, because we perform one operation per character in s. The space complexity is also $O(N)$, as in the worst case, every character in s is pushed onto the stack.

← Previous

Remove All Adjacent Duplicates In String (Easy)

Next →

Minimum Depth Of A Binary Tree (Easy)

☐ Mark as Completed